

Programming Assignment #1 TimeTable (시간표)

Students rely on timetables to record their classes for the week. In this assignment, we will create a timetable app that can allow students to record their class schedules. The student's schedule will be arranged by **days** and **periods**.

Days are from **Monday to Friday** and periods are from **1 to 10**

Create a class named **Subject** that will be used to store the student's subject information.

- Create a variable called **String name** that will be used to store the name of the subject.
- Create a variable called **String tutor** that will be used to store the name of the tutor.
- Create a variable called **String room** that will be used to store the name of the room of the subject.
- Create a constructor that can be used to set all the variables.
- Create a constructor that sets only the name of the subject.
- Create a copy constructor that can be used to create a copy of the subject.
- Create the accessors for getting the **name**, **tutor**, and **room**.
- Create mutators for setting the **tutor** and the **room**.
- Create an equals method that can be used to determine if a subject is equal to another subject. A subject is equal to another if the **name**, **tutor** and the **room** are the same.
- Create a **toString** method that returns the name of the subject.
- Create a method called **getDetails()** that returns the detailed information of the subject
 - Name:
 - Tutor:
 - Room:

Create another class named **TimeTable** that will be used to store the timetable information.

- Create a variable called **DAYS** that will be used to store the days of the week from MONDAY - FRIDAY. The DAYS variable should be an enum type.
- Create a variable called **timeTable** that will be used to store the subjects in a student's timetable. This variable is a multidimensional array that stores Subjects.
- Create a method called **initialize()**. This method is accessible only within the class and is used to initialize the timetable as follows.
 - The array should have 10 rows and 5 columns
 - All the Subjects in row index 3 should be initialized to **BREAK**
 - All the Subjects in row index 7 should be initialized to **LUNCH**
 - Otherwise, by default all other cells should be initialized to "----"
- Create a constructor that accepts no parameters. This constructor simply calls the **initialize** method.
- Create a method named **getSchedule**. This method accepts a **DAY** and a **period** as parameters. It returns the **Subject** that the student has at that slot.

- Create a method named **setSchedule**. This method accepts a **DAY**, **period**, **name**, **tutor** and **room** as parameters. It is used to set a particular subject at a specific day and time. This method returns a **true** if successful and **false** if not.
- **NOTE:** The method must not allow a user to set a class during Breaktime or Lunchtime. If the user attempts to do so, return false.
- Create a **toString()** method that returns a formatted String representation of the timetable. {see the sample output}

Create a third class named **TimeTableApp**. This class will have your main method.

In your main method of this class do the following.

- Declare a **Scanner** for accepting input from the keyboard.
- Create a new **TimeTable** object.
- Create a main menu that does the following.
 - (1) Add a class to my time table
 - (2) View the class at a specific period
 - (3) Print the entire time table
 - (4) Exit the program
- **For option 1**
 - use the Scanner to read the **name**, **day** and **period** to add the class.
 - Attempt to add the class using the method **setSchedule(...)**
 - If the add process is successful, then print Class successfully added otherwise print Class was NOT successfully added
- **For option 2**
 - Use the Scanner to read the day and the period to check.
 - Attempt to get the class at that time using **getSchedule()**
 - Print the value returned as follows “At that time you have ...”
- **For option 3**
 - simply print the formatted timetable {Note your timetable should be formatted in *toString()* method of the *timeTable* class}
- **For option 4**
 - end the program. {The program runs until 4 is selected}

Sample output:

```
===== TIMETABLE APP =====
(1) Add a class to my time table
(2) View the class at a specific period
(3) Print the entire time table
(4) Exit the program
Select an option:
```

```
Select an option: 1
Enter Day (MONDAY/TUESDAY/WEDNESDAY/THURSDAY/FRIDAY): TUESDAY
Enter Period (1-10): 1
Enter Subject Name: OOP
Enter Tutor Name: Lee
Enter Room Name: 507
Class successfully added!
```

```
Select an option: 2
Enter Day (MONDAY-FRIDAY) to view: TUESDAY
Enter Period (1-10) to view: 1

At that time you have ...
Name: OOP
Tutor: Lee
Room: 507
```

```
Select an option: 3
```

PERIOD	MONDAY	TUESDAY	WEDNESDAY	THURSDAY	FRIDAY
Period 1	----	OOP	----	----	----
Period 2	----	----	----	----	----
Period 3	----	----	----	----	----
Period 4	BREAK	BREAK	BREAK	BREAK	BREAK
Period 5	----	----	----	----	----
Period 6	----	----	----	----	----
Period 7	----	----	----	----	----
Period 8	LUNCH	LUNCH	LUNCH	LUNCH	LUNCH
Period 9	----	----	----	----	----
Period 10	----	----	----	----	----

Hints:

- The modifier of the **enum DAYS** may be public.
- You can create the main menu using a do—while loop.

Submission:

You must submit **the Source code** and **Design documentation** via the **LMS** under **“Programming Assignment 1”**.

The deadline for submission is April 30.

Please compress all your **.java** files and your written documentation into a **single .zip file**.

The zip file must be named strictly following this format:

학번_이름.zip (e.g., 2026148696_신우진.zip)

Design Documentation

In the documentation, you should include your program architecture design and a description of implementation methodology as well as an explanation of what you did. **Also, you should include screenshots of your program’s output in the document.**

You may include screenshots of the sample inputs as well as different input values that you chose.

Questions & Support:

If you have any questions regarding this assignment, please send an email to shinwoojin@hanyang.ac.kr or visit IT/BT Bld. 402-2.

Scoring Criteria:

- You will get 100 points if the program satisfies all the requested features and runs smoothly.
- Note: **50 Points for Program code** and **50 Points for the Document**.
- For missing features (e.g. not repeatedly asking for input) points will be deducted.
- In case you do not meet the deadline (April 30),
 - 50% of your score will be deducted for a delay within 24 hours.
 - 75% of your score will be deducted for a delay within 48 hours.
 - 0 points will be given for a delay of more than 48 hours.